Lab 3. WORKING WITH PROCESSES

1. **Write a C program to create a child process such that:**

   + The parent process does not have to wait for the child process

   + The parent process waits for the child process to finish before continuing the program.

2. **Giving below C program is incomplete. Complete the program to create a child process:**

   - *Inside child process:*
     - Print value of two variables **idata** and **istack**. Change value of two variables **idata** and **istack**. Then print the new value of **idata** and **istack** on screen.
     - Change file offset to 7th byte from the beginning of file.
   - *Inside parent process:*
     - Wait for child process to finish.
     - Print value of **idata** and **istack** on screen.
     - Read the content of file *Hello.txt.*
     -

   How are the values of the variables **idata** and **istack** printed in the parent and child processes? Explain why.

   Inside parent process, how is the content of the *Hello.txt* file printed? From there, what conclusions can be drawn about the file descriptor between the parent process and the child process?

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

static int idata = 1000;     //Alocated in data segment
int main() {
   int istack = 150;          //Alocated in stack segment
   int fd = open("output.txt", O_WRONLY);
   pid_t childPid;
   ……
}
```

3. **Write a program to create a child process, which executes ls command to list contents of user's home directory.**

4. **Complete below C program that simulates a shell (parent process) that operates simple shell commands.**

```
#include <sys/types.h>
 #include <sys/wait.h>
#include <unistd.h>
#include <string.h>
#define CMDSIZ 32 ;
extern char **environ int main(int argc, char *argv[] )
{
   int logout=O, cmdsiz;
    char cmdbuf[CMDSIZ] ;
    while(!logout)
   {
      write (1, "myshell> ", 9) ;
      cmdsiz = read(0, cmdbuf, CMDSIZ) ;
      cmdbuf[cmdsiz-1] = '\0' ;
     if (strcmp("logout", cmdbuf) == 0)
            ++logout ;
    else process_command(cmdbuf) ;
} }

void process_command(char* cmdbuf)
{

}
```

Hint: *process_command* function create a child process to execute program like *linuxls, linuxcd* (programs simulate shell commands) in previous lab.

5. **Write a C program in which parent and child processes exchange messages:**

- The child process sends a message "Hello parent" to the parent process. The parent process reads and prints messages from the child process to the screen.

- The parent process sends a message “Hello children” to the child process. The child process reads and prints messages from the parent process to the screen. *Hint:* Using two pipes.

6. **Write a C program that describes the operation of the pipe command: ls | wc -l, where:**
   + The child process executes the ls command
   + The parent process executes the command wc – l
   Hint: using pipe system call to create a pipe between two processes; dup2 system call to redirect input and output.

7. **Write a C-Unix program that continuously checks every 1 minute and prints a list of internal files inside a folder. When CTRL-C is pressed, print messege “Program is terminated by user” on screen and terminate the program.**
   Hint:

- Use opendir, readdir, closeir, stat functions to browse directories and print files inside directories.
- Write a function to handle when an interrupt signal is received (user presses Ctrl + C)